



MongoDB Practicals

Core — CRUD, Queries, Schema, Aggregation & Indexing

Day 2 · hands-on, live on sample_mflix in your Atlas cluster

Where We Are

Day 1 got everyone onto a live Atlas cluster with sample_mflix and stopped right after find(). Today we finish the practical toolkit.

Insert

insertOne · insertMany

Query

operators · AND / \$or · arrays · dot notation

Update

\$set · \$inc · \$unset

Delete

deleteOne · deleteMany

Shape results

projection · sort · limit · skip

Schema

embedding vs referencing

Aggregation

the pipeline · \$lookup

Indexing

createIndex · explain()

Inserting Documents

insertOne, insertMany — and why there is no fixed schema

Inserting Documents

- `insertOne({ ... })` — adds a single document.
- `insertMany([ { ... }, { ... } ])` — adds many at once (an array).
- No `_id`? MongoDB auto-generates a unique `ObjectId`. You almost never set it yourself.
- `movies` already holds ~21,000 real films — you are ADDING your own picks, not building data.
- Tag every test doc with `custom: true` so it is trivial to find and clean up later.

```
db.movies.insertOne({  
  title: "My Indie Pick",  
  year: 2021,  
  genres: ["Drama"],  
  runtime: 96,  
  custom: true  
})
```

Deprecated — do not use

Older tutorials use `insert()`. It is gone from modern drivers. Use `insertOne` / `insertMany` only.

insertMany — many at once

Pass an array of documents. Mix genres and years so later query tasks have variety.

```
db.movies.insertMany([
  { title: "Pick Two", year: 2019,
    genres: ["Comedy"], runtime: 88, custom: true },
  { title: "Pick Three", year: 2020,
    genres: ["Action", "Thriller"], custom: true }
])
```

No fixed schema

“Pick Three” has no runtime and no cast field — and that is completely valid. Two documents in the same collection need not share the same fields.

Verify your own inserts, not the whole collection: `db.movies.countDocuments({ custom: true })`

Querying Documents

find, findOne, operators — the biggest module of the day

find & findOne

- find(filter) returns ALL matches (a cursor). findOne(filter) returns just the first.
- An empty filter {} matches everything: find({}) = “give me all”.
- { rated: "PG" } means the field equals PG.
- For anything other than equals, use an operator inside the field.

```
db.movies.find({ rated: "PG" })

db.movies.findOne(
  { title: "The Godfather" })

db.movies.find({}) // all
```

Meet the cursor

movies has tens of thousands of docs. find() auto-prints only the first 20 in mongosh — type it to see more. That is a cursor, not a bug.

Comparison Operators

Wrap the value in an operator to ask for more than “equals”.

```
// >= 8.5 rating
db.movies.find({ "imdb.rating": { $gte: 8.5 } })

// < 90 minutes
db.movies.find({ runtime: { $lt: 90 } })

// value in a list
db.movies.find({ year: { $in: [1994, 1999] } })

// not equal
db.movies.find({ rated: { $ne: "R" } })
```

The operators

\$gt greater than

\$gte $\geq$

\$lt less than

\$lte $\leq$

\$ne not equal

\$in any of a list

eq equal

NOTE: In JavaScript/JSON syntax, unquoted object keys must be valid identifiers (letters, numbers, underscores, and \$ signs). A period (.) is treated as an operator by the JavaScript parser, not a literal character in an identifier name.

With quotes: "imdb.rating" is read as a literal string key. MongoDB's query engine intercepts strings containing . and interprets them as a path traversing nested fields or arrays (dot notation)

Implicit AND vs \$or

Listing several fields means AND. For OR you must use the \$or operator.

```
// implicit AND – both conditions must hold
db.movies.find({
  genres: "Comedy",
  year: { $gt: 2010 }
})

// OR – either condition may hold
db.movies.find({ $or: [
  { rated: "PG" },
  { "imdb.rating": { $gte: 9 } }
] })
```

Commas in one filter object = AND. \$or takes an array of filter objects and matches if ANY of them do.

Matching Inside Arrays

genres and cast are arrays. Query them by a plain value — it matches if the value is anywhere in the array.

```
// any movie whose genres array contains Comedy  
db.movies.find({ genres: "Comedy" })  
  
// any movie featuring this actor  
db.movies.find({ cast: "Tom Hanks" })  
  
// must contain ALL of these - $all  
db.movies.find({ genres: { $all: ["Drama", "Romance"] } })
```

A plain value on an array field = “contains”. Use \$all when every listed value must be present.

Nested Fields — Dot Notation

imdb is an object. Reach a field inside it with a quoted “parent.child” path.

```
// imdb.rating lives inside the imdb object  
db.movies.find({ "imdb.rating": { $gte: 8 } })  
  
// combine a nested filter with a projection  
db.movies.find(  
  { "imdb.votes": { $gt: 500000 } },  
  { title: 1, "imdb.rating": 1, _id: 0 }  
)
```

Sanity-check any filter before trusting it: `db.movies.countDocuments(filter)`. Same filter also works in the Atlas Data Explorer FILTER bar.

Updating Documents

\$set, \$inc, \$unset — and the habit that saves you

Updating Documents

- `updateOne(filter, change)` changes the FIRST match; `updateMany` changes ALL matches.
- The change almost always uses `$set` to set specific fields.
- `$inc` adds to a number; `$unset` removes a field.
- Safety habit: before any `updateMany`, run `find()` with the SAME filter first.

```
db.movies.updateOne(  
  { title: "The Godfather" },  
  { $set: { watched: true } }  
)
```

The classic beginner mistake

Without an operator like `$set`, MongoDB REPLACES the whole document with what you passed. Always wrap changes in `$set`.

\$inc and \$unset

Use these on your OWN custom: true documents — never change a real movie's numbers.

```
// $inc - add to a number
db.movies.updateOne(
  { title: "My Indie Pick", custom: true },
  { $inc: { runtime: 10 } }
)

// $unset - remove a field entirely
db.movies.updateOne(
  { title: "My Indie Pick", custom: true },
  { $unset: { watched: "" } } // value in the "" doesn't matter
)
```

Ground rule: on REAL movies, only ADD fields. Reserve \$inc / \$unset / overwrites for data you tagged custom: true.

updateMany — look before you leap

updateMany rewrites every matching document. Preview the match set first, then apply.

```
// 1) SEE exactly what you are about to touch  
db.movies.find({ genres: "Comedy" })  
  
// 2) then change all of them in one call  
db.movies.updateMany(  
  { genres: "Comedy" },  
  { $set: { recommended_by_class: true } }  
)
```

Safety habit

find() first with the SAME filter.

A one-second check turns “oops, I changed 4,000 docs” into “yes, that is the set I meant.”

Deleting Documents

deleteOne removes the first match; deleteMany removes all matches. Same look-before-you-leap habit.

```
// check the exact document first...  
db.movies.findOne({ title: "Pick Two", custom: true })  
  
// ...then remove just that one  
db.movies.deleteOne({ title: "Pick Two", custom: true })
```

Deprecated

Old tutorials use remove(). Modern MongoDB uses deleteOne / deleteMany only.

Only ever bulk-delete data YOU created (custom: true). The real sample data stays intact for aggregation work.

Safe Cleanup with deleteMany

Your custom: true tag makes cleaning up your own test data a one-liner.

```
// preview the blast radius  
db.movies.find({ custom: true })  
  
// remove ALL of your test data at once  
db.movies.deleteMany({ custom: true })  
  
// confirm it is gone  
db.movies.countDocuments({ custom: true }) // -> 0
```

If anything ever goes wrong, you can drop sample_mflix and reload the sample dataset from Atlas for a clean slate.

Shaping the Results

projection, sort, limit, skip — HOW results come back

Projection — choose the fields

A second argument to `find()` picks which fields come back. 1 = include, 0 = exclude. `_id` shows unless you hide it.

```
// only title and year; hide the _id  
db.movies.find(  
  { genres: "Comedy" },  
  { title: 1, year: 1, _id: 0 }  
)
```

Rule

Use 1s to build an include list, or 0s to build an exclude list — don't mix them (except `_id: 0`, which is always allowed).

sort, limit, skip

Chain these onto find(). sort({ f: 1 }) ascending, -1 descending. skip + limit = pagination.

```
// top 5 by rating, highest first  
db.movies.find({}, { title: 1, "imdb.rating": 1, _id: 0 })  
  .sort({ "imdb.rating": -1 })  
  .limit(5)  
  
// page 2 of Comedy, newest first, 5 per page  
db.movies.find({ genres: "Comedy" })  
  .sort({ year: -1 })  
  .skip(5)  
  .limit(5)
```

Pagination

Page 1: skip(0)

Page 2: skip(5)

Page 3: skip(10)

...with limit(5) fixed. skip(n) jumps over the first n results.

Putting It All Together

Filter, project, sort and limit compose into one readable query.

```
db.movies.find(  
  { genres: "Drama", "imdb.rating": { $gte: 8 } },  
  { title: 1, year: 1, "imdb.rating": 1, _id: 0 }  
)  
  .sort({ "imdb.rating": -1 })  
  .limit(10)
```

Read it left to right: WHICH documents (filter) · WHICH fields (projection) · in WHAT order (sort) · HOW many (limit).

Schema Design

Embedding vs referencing — the one “think differently from SQL” idea

Embedding vs Referencing

EMBEDDING

```
// genres & cast live INSIDE  
// each movie document  
{ title: "The Godfather",  
  genres: ["Crime", "Drama"],  
  cast: ["Marlon Brando", ...] }
```

One read gets everything.

REFERENCING

```
// a comment points at a movie  
// by its _id (like a foreign key)  
{ movie_id: ObjectId("573a13..."),  
  name: "Mercedes Tyler",  
  text: "Ea quis... " }
```

comments references movies by movie_id.

Rule of thumb

Embed when the related data is small, belongs to this one parent, and is always read together (genres, cast). Reference when it is large, shared, or grows without bound (a blockbuster's comments could be thousands).

The Embedding You Already Have

Every movie carries its genres and cast arrays inside it — that is embedding, and one read returns all of it.

```
db.movies.findOne(  
  { title: "The Godfather" },  
  { title: 1, genres: 1, cast: 1, _id: 0 }  
)  
  
// -> { title, genres: [...], cast: [...] }  
//    no join, no second query
```

In SQL this one movie would be spread across movies + genres + cast tables and joined back together. In MongoDB it is one document.

Referencing + a Manual Join

comments keeps each comment in its own collection and links to a movie by `movie_id`. Follow the link yourself — a preview of \$lookup.

```
// 1) grab any comment and read its movie_id  
const c = db.comments.findOne()  
  
// 2) follow the reference into movies  
db.movies.findOne({ _id: c.movie_id },  
                  { title: 1 })
```

Why not embed comments?

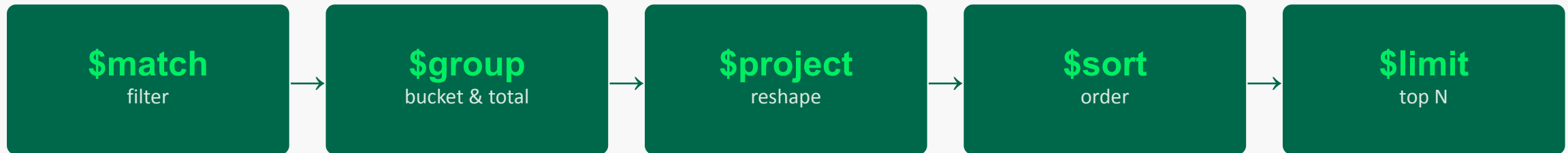
A document has a hard 16 MB limit. A popular film could gather thousands of comments — embedding them risks the ceiling. Referencing avoids it entirely.

Aggregation

The pipeline — process documents in stages

The Aggregation Pipeline

A pipeline is an array of stages. Each stage transforms the stream of documents and passes the result to the next — like a Unix pipe.



Other stages you will meet today

\$skip (jump ahead) · \$unwind (split an array into one document per element) · \$lookup (join another collection). Stage order matters: filter early with \$match so later stages do less work.

\$match + \$group

\$match filters (like find); \$group buckets documents by a key and computes totals with accumulators.

```
// average IMDb rating per certificate
db.movies.aggregate([
  { $match: { "imdb.rating": { $type: "number" } } },
  { $group: {
    _id: "$rated",
    avgRating: { $avg: "$imdb.rating" },
    count: { $sum: 1 }
  } }
])
```

Accumulators

\$sum · \$avg
\$min · \$max
\$push · \$addToSet

A leading "\$" means “the value of this field”.

\$project, \$sort, \$limit, \$skip

Reshape each document, then order and page — the pipeline versions of projection / sort / limit / skip.

```
db.movies.aggregate([
  { $match: { year: { $gte: 2000 } } },
  { $project: { _id: 0, title: 1, year: 1,
               rating: "$imdb.rating" } },
  { $sort: { rating: -1 } },
  { $skip: 0 },
  { $limit: 5 }
])
```

\$project can also rename or compute fields (rating: "\$imdb.rating"), which a find() projection cannot do.

\$unwind — one row per array element

genres is an array. \$unwind expands each movie into one document per genre so you can group by it.

```
// how many movies in each genre?  
db.movies.aggregate([  
  { $unwind: "$genres" },  
  { $group: { _id: "$genres", total: { $sum: 1 } } },  
  { $sort: { total: -1 } }  
])
```

Before / after

1 movie with
genres: [A, B, C]

↓ \$unwind

3 documents, one
per genre.

A Multi-Stage Pipeline

Stages compose: filter → explode the array → group & count → order → take the top 5.

```
// top genres among highly-rated modern films
db.movies.aggregate([
  { $match: { year: { $gte: 2010 },
               "imdb.rating": { $gte: 8 } } },
  { $unwind: "$genres" },
  { $group: { _id: "$genres", films: { $sum: 1 } } },
  { $sort: { films: -1 } },
  { $limit: 5 }
])
```

Match first, then unwind. Filtering early means fewer documents flow through the expensive stages.

\$lookup — a join across collections

\$lookup pulls in matching documents from another collection — the aggregation answer to referencing.

```
// attach each movie's comments, then count them
db.movies.aggregate([
  { $match: { title: "The Godfather" } },
  { $lookup: {
    from: "comments",
    localField: "_id",
    foreignField: "movie_id",
    as: "comments" } },
  { $project: { title: 1,
    commentCount: { $size: "$comments" } } }
])
```

localField (movies._id) is matched against foreignField (comments.movie_id); matches land in the new “comments” array.

Indexing

Why queries get slow — and how to make them fast

Why Indexes Matter

- Without an index, a query scans EVERY document to find matches — a collection scan (COLLSCAN).
- An index is a sorted structure that lets MongoDB jump straight to matching documents (IXSCAN).
- Indexes speed up reads but cost a little on every write and use disk — so index what you actually query.
- `_id` is always indexed for you.

COLLSCAN

reads all
~21,000 docs

IXSCAN

jumps to the
few matches

The trade-off

An index is like a book's index: it makes look-ups fast, but every edit to the book means updating the index too. Worth it for fields you filter or sort on often.

createIndex

Build an index on the fields you query most. 1 = ascending, -1 = descending.

```
// this filter scans every document today  
db.movies.find({ title: "The Godfather" })  
  
// build an index on title  
db.movies.createIndex({ title: 1 })  
  
// see every index on the collection  
db.movies.getIndexes()
```

After creating the index, the same title query jumps straight to its match instead of scanning ~21,000 documents.

explain() — did the index help?

explain("executionStats") shows how MongoDB actually ran the query.

```
db.movies.find({ title: "The Godfather" })  
  .explain("executionStats")  
  
// read winningPlan.stage:  
//   COLLSCAN -> whole collection scanned (slow)  
//   IXSCAN   -> index used (fast)  
//  
// compare totalDocsExamined vs nReturned:  
// close numbers = an efficient query
```

Rule of thumb: if totalDocsExamined is far larger than nReturned, the query is doing too much work — add or fix an index.

You can now drive MongoDB by hand

Insert · query with operators · update & delete safely · shape results · design a schema · aggregate · index.

Next deck — Advanced: replaceOne / bulkWrite / upsert, \$facet & \$bucket, compound & text indexes, and schema patterns.